

Mathematical Modeling of Alqueva PSP-PV-BESS 24-Hour Energy Trading Optimizer

This document presents the complete mathematical formulation of the optimization, physical, and decision-rule models implemented in the production codebase. Equations are extracted directly from the Pyomo source code with no derivation or addition beyond what is implemented. Equation numbering is sequential and continuous across the entire document. Each equation cites its source file and function for direct traceability.

Glossary of Market, Regulatory, and Technical Terms

A reader unfamiliar with European electricity market structure or the plant's physical layout should read this section first; every term below is used without re-expansion throughout the rest of the document.

The plant. The Alqueva hybrid asset combines three technologies behind one grid connection: a pumped-storage hydropower (**PSP**) plant with 4 reversible Francis units (each can either generate by releasing water downhill through a turbine, or pump water back uphill to store energy, but never both at once), a 5 MWp floating solar array (**PV**), and a 1 MW / 2 MWh battery (**BESS**). The PSP moves water between two reservoirs: Alqueva (upper) and Pedrógão (lower). Generating drains the upper reservoir into the lower one; pumping reverses the flow, consuming grid electricity to lift water back up for later use — this is what makes PSP a storage technology, not just hydropower.

Why the plant sells into so many different markets. A single MWh of capacity can be sold multiple times over, in different markets, at different times before delivery, each with a different purpose:

- **Day-Ahead (DA)** — the main auction, closing at noon the day before delivery, where most of the day's energy position is set via the Iberian exchange **OMIE** (Operador del Mercado Ibérico de Energía).

- **Intraday auctions (IDA1, IDA2, IDA3)** — three further auctions on the **SIDC** (Single Day-ahead/Intraday Coupling, the EU-wide intraday auction mechanism), each closing progressively closer to delivery, letting the plant correct its DA position as forecasts improve. Each later auction can only re-trade hours that haven’t already passed their own gate-close.
- **XBID** — continuous intraday trading (not a scheduled auction; orders can be placed any time up to 1 hour before delivery), part of the same SIDC framework.
- **aFRR (automatic Frequency Restoration Reserve)** and **mFRR (manual Frequency Restoration Reserve)** — these are not energy sales but *capacity* sales: the plant promises to hold back some MW of headroom that the grid operator can call on short notice to keep grid frequency stable. aFRR responds automatically and fast (FAT, see below, of 5 minutes) via the European **PICASSO** platform; mFRR responds to a manual TSO instruction and is slower (FAT 12.5 minutes) via the European **MARI** platform.
- **FCR (Frequency Containment Reserve)** — a *mandatory*, unpaid grid-code obligation (not a market) requiring the plant to always keep a small amount of headroom in reserve for primary frequency response; this headroom is subtracted from every other market’s available capacity before anything is offered or sold.

Settlement, after the fact. Once delivery happens, every one of the above commitments is paid or charged based on what actually happened, not what was forecast. **Imbalance settlement** penalizes the plant under “dual pricing” for any gap between what it promised (scheduled) and what it actually delivered (actual) — this is the financial incentive to forecast and dispatch accurately.

The full day, in order. Every part of this document maps onto a single point in this timeline. “D-1” means the day before delivery; “D” means the delivery day itself.

When	Gate / Phase	What happens
D-1, noon	DA (Part 1)	Main day-ahead position is bid for all 24 hours
D-1, 15:00	IDA1 (Part 2A)	First correction, all 24 hours

When	Gate / Phase	What happens
		still open
D-1, 18:30	XBID window W1 (Part 2D)	Continuous trading opens, all hours still open
D-1, 22:00	IDA2 (Part 2B)	Second correction, hours 1–2 now locked
D-1 onward	aFRR / mFRR capacity offers (Parts 3A, 3B)	Reserve capacity offered for the next delivery day
D, 09:30	XBID window W2 (Part 2D)	Continuous trading check, hours ≥ 11 still open
D, 10:00	IDA3 (Part 2C)	Final correction, hours 1–11 now locked
D, during each hour	Real-time dispatch, aFRR/mFRR activation (Parts 4A, 4B, 4C)	The schedule is physically executed; the TSO may call reserve at any moment
After D ends	Settlement (Parts 5A, 5B, 5C)	Every gate and every activation is priced against final cleared/settlement prices
After settlement	Analytics and reporting (Part 5D)	Daily P&L, KPIs, and the Excel report are assembled
Periodically, offline	Backtesting (Part 6)	Many past days are replayed to validate forecast accuracy and compute portfolio risk

Key acronyms and abbreviations used throughout:

Term	Meaning
MILP	Mixed-Integer Linear Program — the optimization model solved each gate; “integer”

Term	Meaning
	because on/off decisions (e.g., is this turbine running) are binary, “linear” because every constraint and the objective are linear or linearized
TSO	Transmission System Operator — the national grid operator (REN, in Portugal) responsible for keeping the grid balanced and stable
REN	Redes Energéticas Nacionais — the Portuguese TSO, the counterparty for FCR/aFRR/mFRR and the source of regulatory price caps and settlement data
FAT	Full Activation Time — the maximum time a reserve product is allowed to take to reach its full promised MW after the TSO’s call
ISP	Imbalance Settlement Period — the time resolution at which actual delivery is measured and settled (15 minutes in Portugal since 19 March 2025; 96 ISPs per day)
SoC	State of Charge — the battery’s current stored energy, as a fraction or absolute MWh of its capacity
GHI	Global Horizontal Irradiance — the standard measure of solar power reaching a flat surface, the primary input to the PV production model
NOCT	Nominal Operating Cell Temperature — a standard PV industry rating used to convert ambient air temperature into the hotter temperature the solar cell itself reaches

Term	Meaning
OU process	Ornstein-Uhlenbeck process — a mathematical model for a quantity that randomly drifts but is pulled back toward a central value over time (mean-reverting), used here to generate realistic synthetic price/spread data for training
McCormick linearization	A standard technique (McCormick, 1976) for handling a product of two unknowns inside a linear solver: rather than solving the nonlinear product directly, four linear inequalities are added that tightly “sandwich” the true value, letting an exact linear solver handle what is really a nonlinear relationship
CET	Central European Time — the timezone used for all market gate-close times referenced in this document
hm ³	Cubic hectometer, the unit used for both reservoirs’ water volume: 1 hm ³ = 10 ⁶ m ³ (one million cubic meters). Reservoir volumes are tracked in hm ³ because the numbers stay in a convenient range (hundreds to thousands) rather than the unwieldy billions of m ³ a large reservoir would otherwise require.
PR- <i>n</i> , INV- <i>n</i> , FR- <i>n</i>	Reference codes (e.g. “PR-4”, “INV-11”) cited occasionally throughout the document. These point back to specific rule numbers in the project’s internal physical/market specification — “PR” for a physical or market prohibition the model must never violate, “INV” for an invariant that must always hold true, “FR” for

Term	Meaning
	a functional requirement. They are bookkeeping cross-references only; no formula or calculation depends on the code itself, so a reader can treat each as a citation tag rather than something to look up.

Part 0 — Common Layer

The common layer holds the single shared 24-hour Mixed-Integer Linear Program (MILP) that serves every market gate (DA, IDA1, IDA2, IDA3), together with the physical plant models, reserve-offer sizing logic, and activation/settlement-input formulas reused across phases. One model serves all gates; gates differ only in their price/forecast inputs and in which hours are frozen to an already-committed position.

C.1 Objective Function

Source: `common_layer/optimisation_model/core_milp_builder.py`, `function _objective`.

The MILP maximizes expected trading profit over the 24-hour horizon, composed of energy revenue, a terminal water-value credit, and four penalty/cost terms (PV curtailment, BESS degradation, reservoir spillage, and turbine startup).

$$\max \quad Z = R_{energy} + V_{water} - C_{pv} - C_{bess} - C_{spill} - C_{start} \quad (1)$$

where Z (EUR) is the objective value, R_{energy} is energy revenue, V_{water} is the terminal water-value credit, and C_{pv} , C_{bess} , C_{spill} , C_{start} are the curtailment, degradation, spillage, and startup cost terms defined below.

Energy revenue values the net plant injection at the day-ahead price for every hour in the horizon.

$$R_{energy} = \sum_{h \in H} \pi_h \cdot p_h^{net} \cdot \Delta t \quad (2)$$

where H is the ordered set of delivery hours in the horizon, π_h (EUR/MWh) is the day-ahead price forecast or cleared price for hour h , p_h^{net} (MW) is the net plant grid injection in hour h (positive when exporting, negative when importing for pumping), and Δt (h) is the duration of one timestep (1.0 for hourly gates).

Terminal water value credits (or debits) the change in upper-reservoir storage between the start and end of the horizon at a fixed marginal water value, converted from volume to energy-equivalent units.

$$V_{water} = \lambda_{water} \cdot \kappa_{hm^3} \cdot (v_{h_N}^{up} - v_0^{up}) \quad (3)$$

where λ_{water} (EUR/MWh) is the configured marginal water value, κ_{hm^3} (MWh/hm³) is the energy-equivalent conversion factor for upper-reservoir volume (computed in Eq. 4), $v_{h_N}^{up}$ (hm³) is the upper-reservoir volume at the final hour of the horizon, and v_0^{up} (hm³) is the upper-reservoir volume at the start of the horizon.

$$\kappa_{hm^3} = \frac{P_{max}^{trb}}{Q_{max}^{trb}} \cdot 10^6 \quad (4)$$

where P_{max}^{trb} (MW) is the rated turbine power per unit at nameplate flow, and Q_{max}^{trb} (m³/h) is the rated turbine flow per unit; the factor 10^6 converts m³ to hm³.

PV curtailment penalty discourages spilling available solar production.

$$C_{pv} = \lambda_{pv} \sum_{h \in H} p_h^{curt} \cdot \Delta t \quad (5)$$

where λ_{pv} (EUR/MWh) is the PV curtailment penalty rate, and p_h^{curt} (MW) is PV power curtailed in hour h .

BESS degradation cost is a cycle-weighted cost applied to every MWh that passes through the battery, whether charged from the grid, charged from PV, or discharged.

$$C_{bess} = \lambda_{deg} \sum_{h \in H} (p_h^{chg} + p_h^{pv \rightarrow bess} + p_h^{dis}) \Delta t \quad (6)$$

where λ_{deg} (EUR/MWh) is the BESS degradation cost rate, p_h^{chg} (MW) is grid-sourced BESS charge power, $p_h^{pv \rightarrow bess}$ (MW) is PV-sourced BESS charge power, and p_h^{dis} (MW) is BESS discharge power.

Reservoir spillage penalty discourages uncontrolled release of water from the upper reservoir.

$$C_{spill} = \lambda_{spill} \sum_{h \in H} q_h^{spill} \cdot \Delta t \quad (7)$$

where λ_{spill} (EUR/m³) is the spillage penalty rate, and q_h^{spill} (m³/h) is the upper-reservoir spill flow in hour h .

Turbine startup cost penalizes every cold start of every unit across the horizon.

$$C_{start} = \lambda_{su} \sum_{u \in U} \sum_{h \in H} z_{u,h}^{su} \quad (8)$$

where λ_{su} (EUR) is the fixed cost per turbine start, U is the set of PSP units (4 reversible Francis units), and $z_{u,h}^{su}$ is a binary variable equal to 1 if unit u starts generating in hour h (defined in Eq. 13).

C.2 PSP Turbine and Pump Constraints

Source: common_layer/optimisation_model/core_milp_builder.py, PSP constraint block (mode_excl, turb_max, turb_min, pump_max, pump_min, turb_start).

Each of the four reversible Francis units may generate (turbine), pump, or sit idle in any hour; the three modes are mutually exclusive.

$$x_{u,h}^{trb} + x_{u,h}^{pmp} \leq 1 \quad \forall u \in U, h \in H \quad (9)$$

where $x_{u,h}^{trb}$ and $x_{u,h}^{pmp}$ are binary variables equal to 1 if unit u is generating, respectively pumping, in hour h .

Turbine output, when the unit is on, must lie between minimum stable load and nameplate capacity; when off, output is forced to zero.

$$P_{min}^{trb} \cdot x_{u,h}^{trb} \leq p_{u,h}^{trb} \leq P_{max}^{trb} \cdot x_{u,h}^{trb} \quad \forall u \in U, h \in H \quad (10)$$

where $p_{u,h}^{trb}$ (MW) is the turbine power of unit u in hour h , and P_{min}^{trb} , P_{max}^{trb} (MW) are the minimum stable load and nameplate capacity per unit. Summed over all n_U units, this constraint also bounds total fleet turbine output at $n_U \cdot P_{max}^{trb}$, the plant's nameplate generation capacity (PR-4).

Pump intake, when the unit is pumping, must lie between a derived minimum power and rated pump power.

$$P_{min}^{pmp} \cdot x_{u,h}^{pmp} \leq p_{u,h}^{pmp} \leq P_{max}^{pmp} \cdot x_{u,h}^{pmp} \quad \forall u \in U, h \in H \quad (11)$$

where $p_{u,h}^{pmp}$ (MW) is the pump power of unit u in hour h , P_{max}^{pmp} (MW) is rated pump power, and P_{min}^{pmp} is derived from the minimum pump flow fraction:

$$P_{min}^{pmp} = P_{max}^{pmp} \cdot \frac{Q_{min}^{pmp}}{Q_{max}^{pmp}} \quad (12)$$

where Q_{min}^{pmp} , Q_{max}^{pmp} (m³/h) are the minimum and maximum pump flow rates.

A turbine start is detected whenever the on-state transitions from off to on between consecutive hours; for the first hour of the horizon, any on-state at h_1 counts as a start.

$$z_{u,h}^{su} \geq \begin{cases} x_{u,h}^{trb} & h = h_1 \\ x_{u,h}^{trb} - x_{u,h-1}^{trb} & h \neq h_1 \end{cases} \quad \forall u \in U, h \in H \quad (13)$$

where h_1 denotes the first hour of the horizon and $h - 1$ denotes the immediately preceding hour in H .

C.3 Head-Volume Relationship and McCormick Linearization

Source: common_layer/optimisation_model/core_milp_builder.py, constraints head_vol, mc_trb, mc_pmp.

The hydraulic head is modeled as a linear function of upper-reservoir volume, calibrated to the plant's operating range.

$$H_h^{net} = H_{min} + \frac{dH}{dV} (v_h^{up} \cdot 10^6 - V_{ref}) \quad \forall h \in H \quad (14)$$

where H_h^{net} (m) is the net hydraulic head in hour h , $H_{min} = 54.7$ m and $H_{max} = 73.0$ m are the operating head bounds, v_h^{up} (hm³) is upper-reservoir volume in hour h , V_{ref} (m³) is the reservoir volume corresponding to H_{min} , and $\frac{dH}{dV} = \frac{H_{max}-H_{min}}{V_{range}}$ is the constant head-volume slope, with V_{range} (m³) the usable volume range between minimum and usable upper-reservoir capacity.

Because turbine and pump power depend on the product of head and on/off status (a bilinear term), the model introduces an auxiliary variable $H_{u,h}^{trb} \approx H_h^{net} \cdot x_{u,h}^{trb}$ and linearizes it with the four standard McCormick envelope inequalities (McCormick, 1976):

$$\begin{aligned} H_{u,h}^{trb} &\leq H_{max} \cdot x_{u,h}^{trb} \\ H_{u,h}^{trb} &\geq H_{min} \cdot x_{u,h}^{trb} \\ H_{u,h}^{trb} &\leq H_h^{net} - H_{min}(1 - x_{u,h}^{trb}) \\ H_{u,h}^{trb} &\geq H_h^{net} - H_{max}(1 - x_{u,h}^{trb}) \end{aligned} \quad \forall u \in U, h \in H \quad (15)$$

where $H_{u,h}^{trb}$ (m) is the McCormick auxiliary representing the active head seen by unit u while turbinning in hour h . An identical set of four constraints, denoted $H_{u,h}^{pmp}$, is applied for the pumping mode using $x_{u,h}^{pmp}$ in place of $x_{u,h}^{trb}$.

C.4 Bilinear Efficiency Surface Interpolation

Source: `common_layer/optimisation_model/core_milp_builder.py`, function `_efficiency_surface` and the `omega_*` constraint block.

Turbine and pump efficiency is not constant; it varies with flow and head according to a quadratic response surface fitted over a 5×5 grid of normalized flow and head values.

$$\eta(f_n, h_n) = a_0 + a_1 f_n + a_2 h_n + a_3 f_n h_n + a_4 f_n^2 + a_5 h_n^2 \quad (16)$$

where η is unit efficiency, $f_n, h_n \in [0,1]$ are normalized flow and head coordinates on the grid, and $a_0, \dots, a_5$ are fitted polynomial coefficients (separate coefficient sets for turbine and pump mode). The result is clipped to the physically realistic bound $\eta \in [0.85, 0.92]$.

The grid-cell power coefficient, precomputed once per grid cell from the efficiency surface, converts flow and head into electrical power.

$$c_{f,k}^{trb} = \eta_{f,k} \cdot \rho \cdot g \cdot Q_f^{trb} \cdot H_k / \Gamma \quad (17)$$

$$c_{f,k}^{pmp} = \frac{1}{\eta_{f,k}} \cdot \rho \cdot g \cdot Q_f^{pmp} \cdot H_k / \Gamma \quad (18)$$

where $c_{f,k}^{trb}$, $c_{f,k}^{pmp}$ (MW) are the precomputed power coefficients at flow grid index f and head grid index k , $\eta_{f,k}$ is the corresponding efficiency from Eq. 16, $\rho = 1000 \text{ kg/m}^3$ is water density, $g = 9.81 \text{ m/s}^2$ is gravitational acceleration, Q_f^{trb} , Q_f^{pmp} (m^3/h) are the turbine and pump flow grid points, H_k (m) is the head grid point, and $\Gamma = 3.6 \times 10^9$ is the unit conversion constant from $\text{m}^3/\text{h} \cdot \text{m}$ to MW.

A bilinear interpolation weight $\omega_{u,f,k,h} \geq 0$ is defined per unit, per grid cell, per hour, for both turbine and pump modes — intuitively, ω measures how much each of the 25 grid cells (5 flow points $\times$ 5 head points) “contributes” to the unit’s actual operating point at hour h , the way mixing paint colors in different proportions produces an in-between shade. The weights must sum to the unit’s on-status (zero when off, one when on), which forces them to act as a weighted average (a convex combination, meaning every weight is non-negative and they sum to exactly 1) over the grid cells rather than an arbitrary unconstrained mix.

$$\sum_{f \in F} \sum_{k \in K} \omega_{u,f,k,h}^{trb} = x_{u,h}^{trb} \quad \forall u \in U, h \in H \quad (19)$$

where F and K are the sets of flow and head grid indices (each of size 5). An identical constraint applies to $\omega_{u,f,k,h}^{pmp}$ against $x_{u,h}^{pmp}$.

The interpolated weights recover turbine and pump power, flow, and active head through four linear identities:

$$p_{u,h}^{trb} = \sum_{f,k} \omega_{u,f,k,h}^{trb} \cdot c_{f,k}^{trb} \quad (20)$$

$$q_{u,h}^{trb} = \sum_{f,k} \omega_{u,f,k,h}^{trb} \cdot Q_f^{trb} \quad (21)$$

$$H_{u,h}^{trb} = \sum_{f,k} \omega_{u,f,k,h}^{trb} \cdot H_k \quad (22)$$

where $q_{u,h}^{trb}$ (m³/h) is the turbine flow of unit u in hour h used in the reservoir water balance, and $H_{u,h}^{trb}$ is the McCormick auxiliary from Eq. 15, closing the link between the efficiency surface and the head model. Three analogous constraints (power, flow, head) apply to the pump mode using $\omega_{u,f,k,h}^{pmp}$, $c_{f,k}^{pmp}$, and $p_{u,h}^{pmp}$, $q_{u,h}^{pmp}$, $H_{u,h}^{pmp}$.

C.5 BESS Constraints

Source: common_layer/optimisation_model/core_milp_builder.py, constraints bess_excl, chg_cap, dis_cap, pv_to_bess_cap, soc_balance, soc_lo, soc_hi.

The battery cannot charge and discharge in the same hour.

$$y_h^{chg} + y_h^{dis} \leq 1 \quad \forall h \in H \quad (23)$$

where y_h^{chg} , y_h^{dis} are binary variables equal to 1 if the BESS is charging, respectively discharging, in hour h .

Total charge power, summing the grid-sourced and PV-sourced paths, and discharge power, are each capped at rated BESS power when active.

$$p_h^{chg} + p_h^{pv \rightarrow bess} \leq p^{bess} \cdot y_h^{chg} \quad \forall h \in H \quad (24)$$

$$p_h^{dis} \leq p^{bess} \cdot y_h^{dis} \quad \forall h \in H \quad (25)$$

where p^{bess} (MW) is the BESS rated power (1.0 MW).

State of charge evolves with round-trip efficiency applied separately on the charge and discharge legs, combining both charge sources.

$$e_h = e_{h-1} + \eta_c (p_h^{chg} + p_h^{pv \rightarrow bess}) \Delta t - \frac{p_h^{dis} \cdot \Delta t}{\eta_d} \quad \forall h \in H \quad (26)$$

where e_h (MWh) is BESS stored energy at the end of hour h (with e_{h_1-1} taken as the configured initial state of charge), and η_c , η_d are the charging and discharging round-trip efficiencies.

Stored energy is bounded between configured minimum and maximum levels at all times.

$$E_{min} \leq e_h \leq E_{max} \quad \forall h \in H \quad (27)$$

where E_{min} , E_{max} (MWh) are the minimum and maximum allowable BESS energy.

C.6 PV Energy Balance

Source: common_layer/optimisation_model/core_milp_builder.py, constraint pv_balance.

Available PV generation in every hour is fully allocated across three uses: delivered to the grid, routed internally to charge the BESS, or curtailed.

$$p_h^{pv} + p_h^{pv \rightarrow bess} + p_h^{curt} = p_h^{av} \quad \forall h \in H \quad (28)$$

where p_h^{pv} (MW) is PV power delivered to the grid in hour h , and p_h^{av} (MW) is PV power physically available in hour h (computed by the PV production model, Eq. 56).

C.7 Reservoir Dynamics

Source: common_layer/optimisation_model/core_milp_builder.py, constraints vup_balance, vlow_balance, vup_lo/hi, vlow_lo/hi, terminal_reservoir.

Water moves between the two reservoirs of the closed-loop system: turbinning moves water from the upper reservoir (Alqueva) to the lower reservoir (Pedrógão); pumping reverses the direction. Natural inflow enters the upper reservoir only, and spill leaves the upper reservoir out of the system.

$$v_h^{up} = v_{h-1}^{up} + \frac{\Delta t}{10^6} \left(I_h + \sum_{u \in U} q_{u,h}^{pmp} - \sum_{u \in U} q_{u,h}^{trb} - q_h^{spill} \right) \quad \forall h \in H \quad (29)$$

$$v_h^{low} = v_{h-1}^{low} + \frac{\Delta t}{10^6} \left(\sum_{u \in U} q_{u,h}^{trb} - \sum_{u \in U} q_{u,h}^{pmp} \right) \quad \forall h \in H \quad (30)$$

where v_h^{up} , v_h^{low} (hm³) are upper- and lower-reservoir volumes at the end of hour h (with $v_{h_1-1}^{up}$, $v_{h_1-1}^{low}$ taken as the configured initial volumes), and I_h (m³/h) is the forecast or measured natural inflow into the upper reservoir in hour h .

Both reservoirs are bounded between configured operating limits.

$$V_{min}^{up} \leq v_h^{up} \leq V_{max}^{up} \quad \forall h \in H \quad (31)$$

$$V_{min}^{low} \leq v_h^{low} \leq V_{max}^{low} \quad \forall h \in H \quad (32)$$

where V_{min}^{up} , V_{max}^{up} (hm³) are the minimum and usable-maximum upper-reservoir volumes, and V_{min}^{low} , V_{max}^{low} (hm³) are the minimum and full-capacity lower-reservoir volumes.

A terminal hard constraint requires the upper reservoir to end the horizon no lower than its starting volume, preventing day-by-day depletion under imperfect water-value calibration.

$$v_{h_N}^{up} \geq v_0^{up} \quad (33)$$

where h_N denotes the final hour of the horizon.

The reservoir-pair conservation invariant — verified independently by the standalone reservoir checker rather than enforced as a solver constraint — confirms that the only net change to the combined two-reservoir volume across a step is natural inflow in and spill out.

$$(v_h^{up} + v_h^{low}) - (v_{h-1}^{up} + v_{h-1}^{low}) = \frac{\Delta t}{10^6} (I_h - q_h^{spill}) \quad (34)$$

Source: common_layer/physical_plant_models/reservoir_model.py, function validate_trajectory.

C.8 Energy Balance and FCR-Reduced Envelope

Source: common_layer/optimisation_model/core_milp_builder.py, constraints pnet_balance, pnet_hi, pnet_lo.

Net plant injection sums the PSP fleet's net output, PV delivered to the grid, and the BESS net contribution. PV routed internally to the BESS does not cross the grid boundary and is excluded.

$$p_h^{net} = \sum_{u \in U} (p_{u,h}^{trb} - p_{u,h}^{mp}) + p_h^{pv} + p_h^{dis} - p_h^{chg} \quad \forall h \in H \quad (35)$$

Net injection is bounded by the plant's generation and pump envelopes, each reduced by the mandatory Frequency Containment Reserve (FCR) headroom, which is never sold and is reserved on both the generation and pump sides.

$$-P_{cap}^{pump} \leq p_h^{net} \leq P_{cap}^{gen} \quad \forall h \in H \quad (36)$$

where $P_{cap}^{gen} = P_{max}^{gen} - F_{FCR}$ and $P_{cap}^{pump} = P_{max}^{pump} - F_{FCR}$, with F_{FCR} (MW) the mandatory FCR headroom reserved on both sides (Source: common_layer/physical_plant_models/fcr_headroom_model.py). The nameplate envelopes sum every asset's contribution to net plant injection:

$$P_{max}^{gen} = n_U \cdot P_{max}^{trb} + P^{peak} + P^{bess} \quad (37)$$

$$P_{max}^{pump} = n_U \cdot P_{max}^{pmp} + P^{bess} \quad (38)$$

where n_U is the number of PSP units (4), P^{peak} (MW) is PV nameplate peak capacity, and P^{bess} (MW) is BESS rated power.

Source: common_layer/configuration/plant_config.py, properties p_max_generation_mw, p_max_pump_mw.

For hours frozen by an intraday gate (Section C.9), net position is held fixed rather than optimized.

$$p_h^{net} = N_h^{fix} \quad \forall h \in H^{frozen} \quad (39)$$

where $H^{frozen} \subseteq H$ is the subset of hours frozen to the previously committed schedule, and N_h^{fix} (MW) is the committed net position carried forward for those hours.

C.9 Intraday Re-Optimization Decision Rule

Source: common_layer/optimisation_model/ida_reoptimiser.py, function reoptimise_ida.

Each intraday auction gate (IDA1, IDA2, IDA3) re-solves the core MILP (Eqs. 1–38) under updated intraday prices, with all hours outside that gate's tradable window frozen via Eq. 39. The candidate re-optimized schedule is accepted only if its expected improvement over holding the

previously committed position exceeds a dynamic threshold — the no-churn rule (PR-14), which prevents re-bidding on market microstructure noise.

$$\Delta_h = p_h^{net,new} - N_h^{fix} \quad \forall h \in H^{trade} \quad (40)$$

where $H^{trade} = H \setminus H^{frozen}$ is the tradable hour window for the current gate, $p_h^{net,new}$ (MW) is the re-optimized net position, and Δ_h (MW) is the per-hour position change.

$$\Phi = \frac{1}{2} \sum_{h \in H^{trade}} |\Delta_h| \Delta t \quad (41)$$

where Φ (MWh) is the one-way traded volume; the factor $1/2$ ensures a pure swap (selling one hour, buying another) is counted once rather than twice.

$$I = \sum_{h \in H^{trade}} \pi_h (p_h^{net,new} - N_h^{fix}) \Delta t \quad (42)$$

where I (EUR) is the expected improvement of the re-optimized schedule over the committed baseline, valued at the gate's intraday price forecast π_h .

$$\tau = \max \left(\tau_{floor}, \frac{\tau_{pct}}{100} \sum_{h \in H^{trade}} |\pi_h \cdot N_h^{fix}| \right) \quad (43)$$

where τ (EUR) is the dynamic re-bid threshold, τ_{floor} (EUR) is a configured absolute floor, and τ_{pct} (%) is a configured percentage applied to the absolute committed-position value in the tradable window, so the threshold auto-scales with the plant's market exposure.

The gate submits the re-optimized schedule only if at least one hour's delta exceeds a configured noise floor and the improvement clears the dynamic threshold; otherwise it holds the committed position unchanged.

$$\text{decision} = \begin{cases} \text{RE-BID} & \exists h: |\Delta_h \Delta t| \geq \delta_{min} \text{ and } I \geq \tau \\ \text{NO-CHANGE} & \text{otherwise} \end{cases} \quad (44)$$

where δ_{min} (MWh) is the configured minimum material delta per hour.

C.10 Reserve Capacity Offer Sizing (aFRR / mFRR)

Source: `common_layer/optimisation_model/reserve_offer_builder.py`, functions `fat_deliverable_mw`, `fat_deliverable_dn_mw`, `build_reserve_offers`, `check_reserve_offers`.

Both aFRR and mFRR capacity offers are sized from the headroom remaining after the energy position is committed — a sequential energy-first, reserve-second approach that makes the no-double-sell rule (PR-11) true by construction.

$$\theta_h^{up} = \max(0, P_{cap}^{gen} - N_h - R_h^{up,prior}) \quad (45)$$

$$\theta_h^{dn} = \max(0, N_h + P_{cap}^{pump} - R_h^{dn,prior}) \quad (46)$$

where θ_h^{up} , θ_h^{dn} (MW) are the up and down headroom available for reserve in hour h , N_h (MW) is the committed net energy position in hour h , and $R_h^{up,prior}$, $R_h^{dn,prior}$ (MW) are MW already committed to a higher-priority reserve product in the same hour (e.g., aFRR ahead of mFRR), subtracted to prevent double-selling across products.

The plant's ramp-based deliverable capacity within the product's Full Activation Time (FAT) caps the offer independently of headroom. In pump mode with a short FAT, a full pump-to-generation mode switch is not guaranteed, so the up-deliverable is limited to ramping the pump to zero plus the instantaneous BESS contribution.

$$\psi^{up}(F, N_h, p_h^{av}) = \begin{cases} \min(|N_h|, R_{psp}F) + P^{bess}\beta(p_h^{av}) & N_h < 0 \text{ and } F < F_{safe} \\ R_{psp}F + P^{bess}\beta(p_h^{av}) & \text{otherwise} \end{cases} \quad (47)$$

where ψ^{up} (MW) is the FAT-deliverable up capacity, F (min) is the product's Full Activation Time, R_{psp} (MW/min) is the PSP fleet's total ramp rate, $F_{safe} = 8.0$ min is the configured minimum FAT for a guaranteed mode switch, and $\beta(p_h^{av}) \in \{0,1\}$ is a PV-availability gate equal to 1 only when PV is generating ($p_h^{av} \geq 0.01$ MW), since BESS up-regulation is sourced from PV-charged energy.

$$\psi^{dn}(F) = R_{psp}F + P^{bess} \quad (48)$$

where ψ^{dn} (MW) is the FAT-deliverable down capacity; the down direction requires no mode switch and is therefore symmetric regardless of starting mode.

The final offer takes the minimum of the headroom, the FAT-deliverable capacity (each scaled by a configured headroom fraction for the product, e.g. mFRR reserves a margin), and the product's contractual maximum offer size.

$$o_h^{up} = \min(\phi \cdot \theta_h^{up}, \psi_h^{up}, O_{max}^{up}) \quad (49)$$

$$o_h^{dn} = \min(\phi \cdot \theta_h^{dn}, \psi_h^{dn}, O_{max}^{dn}) \quad (50)$$

where o_h^{up}, o_h^{dn} (MW) are the final offered up and down reserve capacity for hour h , $\phi \in (0,1]$ is the configured headroom fraction, and $O_{max}^{up}, O_{max}^{dn}$ (MW) are the product's contractual maximum offer sizes.

The offer checker verifies, for every hour, that energy plus all reserve commitments (prior and current) remain within the FCR-reduced envelope (PR-11/INV-6) and within FAT deliverability (PR-12):

$$N_h + R_h^{up,prior} + o_h^{up} \leq P_{cap}^{gen} \quad (51)$$

$$N_h - R_h^{dn,prior} - o_h^{dn} \geq -P_{cap}^{pump} \quad (52)$$

$$o_h^{up} \leq \psi_h^{up} \quad \text{and} \quad o_h^{dn} \leq \psi_h^{dn} \quad (53)$$

C.11 BESS Reserve Deliverability

Source: common_layer/physical_plant_models/bess_model.py, functions afrr_up_deliverable_mw, afrr_dn_deliverable_mw.

Independently of the plant-level FAT cap (Eq. 47–48), the battery's own sustained power contribution to an up-regulation call is limited by usable stored energy above the SoC floor, derated by discharge efficiency and spread over the activation time.

$$\sigma^{up}(e_h) = \min\left(p^{bess}, \frac{\max(0, e_h - E_{min}) \cdot \eta_d}{F_{afrr}/60}\right) \quad (54)$$

where σ^{up} (MW) is the maximum power the battery can sustain for the full aFRR FAT, e_h (MWh) is current BESS stored energy, and F_{afrr} (min) is the aFRR Full Activation Time (PICASSO, 5 min).

$$\sigma^{dn}(e_h) = \min\left(p^{bess}, \frac{\max(0, E_{max} - e_h)/\eta_c}{F_{afrr}/60}\right) \quad (55)$$

where σ^{dn} (MW) is the maximum power the battery can sustain charging for the full aFRR FAT, limited by headroom to the SoC ceiling.

C.12 FAT Ramp Trajectory and Effective Settlement Energy

Source: common_layer/optimisation_model/activation_ramp_tracker.py, functions effective_energy_mwh, effective_isp_hours.

When a reserve activation is called, the BESS responds instantaneously while the PSP fleet ramps linearly from zero to its target contribution over the product's Full Activation Time, then holds flat for the remainder of the Imbalance Settlement Period (ISP). The energy actually delivered during an activation is therefore less than the face-value MW multiplied by the full ISP duration; it is the area under a trapezoid (linear ramp) plus a rectangle (plateau).

$$e^{eff} = M \cdot \frac{D_{isp} - F/2}{60} \quad (56)$$

where e^{eff} (MWh) is the ramp-corrected effective energy delivered by one activation, M (MW) is the target activation power, D_{isp} (min) is the ISP duration, and F (min) is the product's FAT.

$$\Delta t_{isp}^{eff} = \frac{D_{isp} - F/2}{60} \quad (57)$$

where Δt_{isp}^{eff} (h) is the effective ISP duration multiplier used to convert activated MW into settled MWh, replacing the face-value ISP duration in settlement (e.g., aFRR: $\Delta t_{isp}^{eff} = 0.2083$ h vs. face 0.25 h, a 16.7% reduction; mFRR: 0.1458 h vs. face 0.25 h, a 41.7% reduction).

C.13 PV Production Physical Model

Source: common_layer/physical_plant_models/pv_production_model.py, function production_mw.

The PV array's physically available power feeds p_h^{av} in Eq. 28. It converts plane-of-array irradiance and cell temperature into AC power, applying a temperature derate and linear annual degradation since commissioning.

$$p^{av} = \text{clip} \left(P^{peak} \cdot \frac{G}{G_{ref}} \cdot [1 + \gamma(T_{cell} - T_{ref})] \cdot (1 - \delta)^Y, 0, P_{eff}^{peak} \right) \quad (58)$$

where P^{peak} (MW) is nameplate peak capacity, G (W/m²) is plane-of-array irradiance, $G_{ref} = 1000$ W/m² is reference irradiance, γ (°C⁻¹) is the (negative) temperature coefficient, T_{cell} , $T_{ref} = 25$ °C are cell and reference temperature, δ (year⁻¹) is the annual degradation rate, Y (years) is years elapsed since commissioning, and $P_{eff}^{peak} = P^{peak}(1 - \delta)^Y$ is the degraded effective peak capacity that caps the result.

C.14 Post-Solve Checker Flow Model

Source: `common_layer/physical_plant_models/psp_turbine_pump_model.py`, functions `turbine_flow_m3h`, `pump_flow_m3h`.

Independently of the MILP's bilinear efficiency-surface flow (Eq. 21), the post-solve physical checker uses a simpler linear flow approximation to cross-validate dispatch results, interpolating flow linearly between the minimum and maximum operating points.

$$q^{trb}(p) = Q_{min}^{trb} + \frac{p - P_{min}^{trb}}{P_{max}^{trb} - P_{min}^{trb}} (Q_{max}^{trb} - Q_{min}^{trb}) \quad (59)$$

where $q^{trb}(p)$ (m³/h) is checker-estimated turbine flow for a given turbine power p (MW), zero when $p \leq 0$.

$$q^{pmp}(p) = Q_{min}^{pmp} + \frac{p - P_{min}^{pmp}}{P_{max}^{pmp} - P_{min}^{pmp}} (Q_{max}^{pmp} - Q_{min}^{pmp}) \quad (60)$$

where $q^{pmp}(p)$ (m³/h) is checker-estimated pump flow for a given pump power p (MW), using the same derived P_{min}^{pmp} as Eq. 12 so the checker and solver share an identical base point.

C.15 Realized Power-Weighted Efficiency (Post-Solve Reporting)

Source: `common_layer/optimisation_model/core_milp_solver.py`, function `_eta_pw`.

After the MILP is solved, the realized efficiency at each hour is recovered from the solved omega interpolation weights as a power-throughput-weighted average across all active grid cells, giving the true efficiency at the unit's actual operating point rather than a single grid value.

$$\bar{\eta}_h^{trb} = \frac{\sum_{u \in U} \sum_{f,k} \omega_{u,f,k,h}^{trb} \eta_{f,k} Q_f^{trb} H_k}{\sum_{u \in U} \sum_{f,k} \omega_{u,f,k,h}^{trb} Q_f^{trb} H_k} \quad (61)$$

where $\bar{\eta}_h^{trb}$ is the power-weighted realized turbine efficiency for hour h , evaluated from the solved $\omega_{u,f,k,h}^{trb}$ values (Eq. 19–22); the denominator is zero (and $\bar{\eta}_h^{trb}$ reported as zero) when no unit is turbinizing in hour h . An identical formula, denoted $\bar{\eta}_h^{pmp}$, applies to pump-mode efficiency using $\omega_{u,f,k,h}^{pmp}$.

C.16 Reservoir Safety Under Reserve Activation

Source: `common_layer/physical_plant_models/reservoir_activation_checker.py`, function `check_reservoir_during_activation`.

The scheduled dispatch already respects reservoir bounds within the MILP, but TSO-instructed aFRR/mFRR activations add extra generation or pumping on top of the committed schedule, which can move additional water beyond what the optimizer assumed — particularly into the smaller lower (Pedrógão) reservoir. This post-delivery checker re-derives water flow from the committed-plus-activated net power using a proportional (linear) plant-level flow model, distinct from both the MILP’s bilinear surface (Eq. 21) and the per-unit linear checker (Eq. 59–60).

$$q^{flow}(n) = \begin{cases} \frac{\min(n, P_{max}^{gen})}{P_{max}^{gen}} \cdot (n_U \cdot Q_{max}^{trb}) & n \geq 0 \\ -\frac{\min(|n|, P_{max}^{pump})}{P_{max}^{pump}} \cdot (n_U \cdot Q_{max}^{pmp}) & n < 0 \end{cases} \quad (62)$$

where $q^{flow}(n)$ (m³/h) is the proportional upper-to-lower flow rate (positive when generating drains the upper reservoir, negative when pumping drains the lower reservoir) for a given effective net power n (MW, committed schedule plus net activation), and $n_U \cdot Q_{max}^{trb}$, $n_U \cdot Q_{max}^{pmp}$ are the full-fleet turbine and pump flow capacities.

The reservoir pair is then stepped per Imbalance Settlement Period (ISP) using this flow together with the pro-rated natural inflow share for that ISP, and the resulting upper- and lower-reservoir levels are checked against the same bounds as Eq. 31–32.

$$\Delta v_{isp}^{up} = (-q^{flow}(n) \cdot \Delta t_{isp} + I_h/n_{isp,h})/10^6 \quad (63)$$

where Δv_{isp}^{up} (hm³) is the upper-reservoir volume change for one ISP, Δt_{isp} (h) is the ISP duration, and $n_{isp,h}$ is the number of ISPs within hour h , so that $I_h/n_{isp,h}$ pro-rates the hourly natural inflow across ISPs.

Part 1 — Phase 1: Day-Ahead Bidding

Phase 1 forecasts the three exogenous inputs the core MILP needs (DA price, PV availability, reservoir inflow), solves the model from Part 0, runs the bid checker (re-validating the Part 0 invariants against the solved schedule), runs a commercial risk gate, and formats the result as an OMIE bid. This part documents only the equations not already covered in Part 0: the forecasting feature engineering, the physical solar models, the accuracy metrics used for model selection, and the risk-gate thresholds.

1.1 Cyclical Calendar Feature Encoding

Source: `phase_1_da_day_ahead_bidding/da_price_pv_inflow_forecasting/da_price_forecaster.py`, `pv_power_forecaster.py`, `reservoir_inflow_forecaster.py`, function `_build_features` in each.

Calendar variables (hour-of-day, day-of-week, month, day-of-year) are cyclical, so each is encoded as a sine/cosine pair rather than a raw integer, ensuring the model sees period 23 and period 0 as adjacent rather than maximally distant.

$$x_k^{sin} = \sin\left(\frac{2\pi(k - k_0)}{K}\right), \quad x_k^{cos} = \cos\left(\frac{2\pi(k - k_0)}{K}\right) \quad (64)$$

where k is the raw calendar value (hour-of-day, day-of-week, month, or day-of-year), k_0 is the index offset so the cycle starts at zero, and K is the period (24 for hour, 7 for day-of-week, 12 for month, 365 for day-of-year). This same encoding is applied independently to each calendar variable used as a model feature in the DA price, PV (GHI and ambient temperature), and inflow forecasters.

1.2 Lag, Rolling, and Trend Features

Source: `da_price_forecaster.py`, `pv_power_forecaster.py`, `reservoir_inflow_forecaster.py`, function `_build_features` in each.

Each forecaster builds lagged values, rolling statistics, and a short-term trend signal from its own target series, all shifted to avoid leakage from the value being predicted.

$$y_{h,\ell}^{lag} = y_{h-\ell} \quad (65)$$

$$\bar{y}_{h,w}^{roll} = \frac{1}{w} \sum_{i=1}^w y_{h-i}, \quad s_{h,w}^{roll} = \text{std}(y_{h-1}, \dots, y_{h-w}) \quad (66)$$

$$\Delta y_h = y_{h-\ell_1} - y_{h-\ell_2} \quad (67)$$

where y_h is the target series (DA price in EUR/MWh, GHI in W/m², ambient temperature in °C, or daily inflow in m³/h), ℓ is a lag offset (e.g., 24 h, 48 h, 168 h, 336 h for hourly targets; 1, 7, 30 days for the daily inflow target), w is a rolling window length (24 h or 168 h for hourly targets; 7 or 30 days for inflow), and Δy_h is the short-term trend feature, the difference between two lagged values ($\ell_1 = 24\text{h}$, $\ell_2 = 48\text{h}$ for DA price; 1-day lag for inflow).

1.3 Model Accuracy Metrics

Source:

phase_1_da_day_ahead_bidding/da_price_pv_inflow_forecasting/ml_train_val_test_common.py,
functions mae, metrics.

Four metrics quantify forecast accuracy and are used both for offline evaluation and for the walk-forward model-selection rule (Eq. 70).

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i| \quad (68)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2} \quad (69)$$

$$\text{Bias} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) \quad (70)$$

$$\text{Skill} = 1 - \frac{\text{MAE}}{\text{MAE}_{naive}} \quad (71)$$

where $\hat{y}_i$, y_i are the predicted and actual values for sample i , n is the number of evaluation samples, and MAE_{naive} is the MAE of the naive (persistence) baseline, so Skill expresses the fractional accuracy improvement of a model over simple persistence.

1.4 Walk-Forward Cross-Validation and Model Selection

Source: `ml_train_val_test_common.py`, function `walk_forward_cv`; auto-selection logic in each forecaster's `_auto_select_model`.

Picking a model by how well it fits the data it was trained on is misleading — a flexible model can simply memorize the training data and still perform badly on a new, unseen day. Cross-validation avoids this trap by testing each candidate model on data it never saw during training, so the comparison reflects genuine forecasting skill rather than memorization. Because price and weather data are time-ordered, a model must never be validated on data that is chronologically *before* the data it trained on (that would leak future information backward); **walk-forward** cross-validation respects this by always training on an earlier block and validating on the very next block in time, sliding forward fold by fold.

Each forecaster (DA price, PV's GHI and ambient temperature, reservoir inflow) auto-selects the best of three candidate models, ranging from simplest to most flexible — **Naive persistence** (tomorrow equals the same hour yesterday, or 7 days ago — the baseline every other model must beat), **Ridge regression** (an ordinary linear model with an added penalty that shrinks coefficients to prevent overfitting on noisy features), and **LightGBM** (a gradient-boosted decision-tree ensemble that can capture nonlinear patterns the linear model cannot, at the cost of being harder to interpret) — via walk-forward cross-validation, re-run only when new training data has arrived since the last selection.

$$n_{fold} = \left\lfloor \frac{n}{K+1} \right\rfloor, \quad \text{fold } k: \text{ train } [1, k \cdot n_{fold}], \text{ validate } (k \cdot n_{fold}, (k+1) \cdot n_{fold}] \quad (72)$$

where n is the total number of historical samples, K is the number of folds (4), and each fold $k = 1, \dots, K$ trains on all data up to the fold boundary and validates on the next block, so no fold ever validates on data the model could have seen in training.

$$m^* = \arg \min_{m \in \{Naive, Ridge, LightGBM\}} \frac{1}{K} \sum_{k=1}^K MAE_{m,k} \quad (73)$$

where m^* is the selected model and $MAE_{m,k}$ (Eq. 68) is model m 's validation MAE on fold k .

1.5 Day-Ahead Price Synthetic Fallback

Source: `da_price_forecaster.py`, function `_synthetic_fallback`.

If the trained model or its historical data is unavailable, the DA price forecaster falls back to a deterministic synthetic curve shaped like typical Iberian price profiles (morning and evening peaks, a midday solar-driven dip, and a night discount), so the pipeline never crashes for lack of data.

$$\pi_h^{synth} = \mu + A \left[e^{-\frac{(h-9)^2}{8}} + 1.15 e^{-\frac{(h-20)^2}{6}} - 0.6 e^{-\frac{(h-14)^2}{10}} + n_h \right] + \epsilon_h \quad (74)$$

where π_h^{synth} (EUR/MWh) is the synthetic price for hour h , $\mu = 55.0$ EUR/MWh is the configured mean price level, $A = 22.0$ EUR/MWh is the configured amplitude, $n_h = -0.8$ for $h \leq 6$ or $h \geq 23$ (night discount) and 0 otherwise, and ϵ_h is uniform noise on $[-3,3]$ EUR/MWh drawn from a delivery-date-seeded random generator (deterministic given the date).

This identical formula appears a second time, independently implemented, in `phase_1_da_day_ahead_bidding/da_price_pv_inflow_forecasting/omie_da_price_loader.py` (function `_synthetic_prices`), used to gap-fill the training Excel file when a live OMIE download fails. Both call sites compute the same Eq. 74; this is noted as a code-duplication observation, not a second equation.

1.6 PV Cell Temperature Model

Source: `phase_1_da_day_ahead_bidding/da_price_pv_inflow_forecasting/pv_power_forecaster.py`, inline in `_model_forecast` and `_synthetic_fallback` (IEC 61215 NOCT model).

Forecasted ambient temperature is converted to PV cell temperature — the temperature that actually drives the PV production model (Eq. 58) using the Nominal Operating Cell Temperature (NOCT) model.

$$T_{cell} = T_{amb} + \frac{NOCT - 20}{800} \cdot G \quad (75)$$

where T_{cell} , T_{amb} (°C) are cell and ambient temperature, $NOCT = 45$ °C is the floating-array Nominal Operating Cell Temperature (IEC 61215), and G (W/m²) is plane-of-array irradiance (GHI).

1.7 Clearness Index

Source: `pv_power_forecaster.py`, function `_build_features`.

The clearness index — actual irradiance normalized by the theoretical clear-sky maximum (Eq. 77–81) — is used as a lagged cloud-cover feature for the GHI forecaster.

$$k_t = \begin{cases} G/G_{clear} & G_{clear} > 10 \\ 0 & \text{otherwise} \end{cases} \quad (76)$$

where k_t is the clearness index, G (W/m²) is measured/historical GHI, and G_{clear} (W/m²) is the theoretical clear-sky GHI for the same hour and location.

1.8 Clear-Sky GHI — Bird Simplified Model

Source: `pv_power_forecaster.py`, function `_clearsky_ghi`.

The clear-sky GHI model provides both the clearness-index feature (Eq. 76) and the basis for the gap-fill and synthetic-fallback irradiance profiles. It computes theoretical irradiance at Alqueva (38.20°N, 7.49°W) from solar geometry and atmospheric transmittance (Bird & Hulstrom, 1981).

Solar declination (Spencer, 1971) and the equation of time are computed from day-of-year:

$$\delta = 0.006918 - 0.399912\cos B + 0.070257\sin B - 0.006758\cos 2B \\ + 0.000907\sin 2B - 0.002697\cos 3B + 0.001480\sin 3B \quad (77)$$

$$E_t = 0.000075 + 0.001868\cos B - 0.032077\sin B - 0.014615\cos 2B \\ - 0.04089\sin 2B \quad (78)$$

where δ (rad) is solar declination, E_t (rad) is the equation of time, and $B = \frac{2\pi(d-1)}{365}$ is the day-angle for day-of-year d .

Solar (true) time, hour angle, and solar elevation follow from declination, equation of time, and the site's geographic longitude:

$$t_{solar} = t_{clock} + \frac{4(\lambda - \lambda_{ref}) + 60 \deg(E_t)}{60}, \quad \omega = 15(t_{solar} - 12) \quad (79)$$

$$\sin \alpha = \max(0, \sin \phi \sin \delta + \cos \phi \cos \delta \cos \omega) \quad (80)$$

where t_{clock} (h) is the UTC clock hour, λ, λ_{ref} ($^\circ$) are site longitude and reference longitude (0°), ω ($^\circ$, converted to radians for use) is the solar hour angle, ϕ (rad) is site latitude, and α is solar elevation angle, clipped at zero (irradiance is zero whenever the sun is below the horizon).

Relative air mass (Kasten & Young, 1989) and the Bird-model transmittance terms determine how much extraterrestrial radiation reaches the surface:

$$m_a = \min\left(\frac{1}{\sin \alpha + 0.50572 (\epsilon + 6.07995)^{-1.6364}}, 38.0\right) \quad (81)$$

$$\tau_r = e^{-0.0903 m_a^{0.84} (1 + m_a - m_a^{1.01})}, \quad \tau_a = 0.935^{m_a}, \quad \tau_w = 0.960^{m_a}, \quad \tau_o = 0.997^{m_a} \quad (82)$$

where m_a is relative air mass, ϵ ($^\circ$) is solar elevation in degrees, and $\tau_r, \tau_a, \tau_w, \tau_o$ are the Rayleigh-scattering, aerosol, water-vapor, and ozone transmittance factors (rural Portugal aerosol loading assumed).

The clear-sky direct and diffuse components, and their sum, complete the model:

$$I_0 = 1367 \left(1 + 0.033 \cos \frac{2\pi d}{365}\right) \quad (83)$$

$$G_{clear} = \text{clip}(0.9662 I_0 \sin \alpha \tau_r \tau_a \tau_w \tau_o + \max(0, 0.2710 I_0 \sin \alpha - 0.2939 I_b), 0, 1200) \quad (84)$$

where I_0 (W/m^2) is extraterrestrial irradiance corrected for Earth-Sun distance, $I_b = 0.9662 I_0 \sin \alpha \tau_r \tau_a \tau_w \tau_o$ (W/m^2) is the direct-beam component appearing inside Eq. 84's diffuse term, and G_{clear} (W/m^2) is the resulting clear-sky GHI, clipped to a physically realistic ceiling of 1200 W/m^2 .

1.9 PV and Inflow Synthetic Fallbacks

Source: `pv_power_forecaster.py`, function `_synthetic_fallback`; `reservoir_inflow_forecaster.py`, function `_climatology_fallback`.

If the PV forecaster’s trained model is unavailable, a deterministic sinusoidal irradiance profile between sunrise and sunset substitutes for the clear-sky model, scaled by a random cloud factor seeded on the delivery date:

$$G_h^{synth} = G_{peak} \cdot \max\left(0, \sin\left(\pi \frac{h - h_{rise}}{h_{set} - h_{rise}}\right)\right) \cdot c \quad (85)$$

where G_h^{synth} (W/m²) is synthetic irradiance for hour h , G_{peak} (W/m²) is a season-dependent peak irradiance (950 in summer, 700 otherwise), h_{rise} , h_{set} are season-dependent sunrise/sunset hours, and c is a uniform cloud factor on [0.75,1.0]. The resulting irradiance feeds Eq. 75 and then the PV production model (Eq. 58), exactly as the trained-model path does.

The inflow forecaster’s fallback is simpler: when the trained model or sufficient history is unavailable, it returns the configured monthly climatological mean inflow directly (Source: `common_layer/configuration/plant_config.py`, method `inflow_for_month`, already defined as I_h in Part 0 Section C.7) — no additional formula beyond that lookup.

1.10 Pre-Trade Commercial Risk Gate

Source: `phase_1_da_day_ahead_bidding/risk_and_bid_validation/pre_trade_risk_checker.py`, method `PreTradeRiskChecker.check`.

Before submission, a commercial risk gate — separate from and downstream of the physical bid checker, which re-validates the Part 0 invariants (Eq. 9–13, 23–34) against the solved schedule and introduces no new equations — enforces two portfolio-level sanity limits independent of any single hour’s physical feasibility.

$$\Lambda = \sum_{h \in H} |p_h^{net}| \Delta t \leq \Lambda_{max} \quad (86)$$

where Λ (MWh) is the day's gross traded volume (sum of absolute net position across all hours) and Λ_{max} (MWh) is a configured daily volume cap (default 15,000 MWh), guarding against a runaway or mis-scaled solution.

$$|R_{energy}| \leq R_{max} \quad (87)$$

where R_{energy} (EUR) is the expected energy revenue from Eq. 2, and R_{max} (EUR) is a configured sanity cap on expected revenue magnitude (default 5,000,000 EUR), catching sign errors or scale blow-ups in the price or position inputs before they reach the trading desk.

Part 2A — Phase 2A: IDA1 (First Intraday Auction)

IDA1 closes D-1 15:00 CET (the afternoon of the day before delivery, three hours after the DA gate). It re-optimizes the full 24-hour horizon (no hours are frozen, since the DA gate is the only prior commitment) against an IDA1-specific price forecast, using the shared engine documented in Part 0, Section C.9 (Eq. 40–44). This part documents only what is specific to IDA1: its price forecasting model and its delta-bid formatting.

2A.1 IDA1 Price Spread Forecasting

Source: `phase_2a_ida1_intraday_auction_1/ida1_price_forecasting/ida1_price_forecaster.py`, function `_model_forecast`.

Rather than forecasting the IDA1 price directly, the model forecasts the spread between IDA1 and DA clearing prices for the same hour, which is a more stable, lower-variance target than the price level itself; the IDA1 price is then recovered by adding the predicted spread back onto the (already-known) DA price.

$$s_h = \pi_h^{IDA1} - \pi_h^{DA} \quad (88)$$

where s_h (EUR/MWh) is the IDA1-DA spread for hour h , the regression target trained on historical SIDC IDA1 clearing prices; π_h^{IDA1} , π_h^{DA} are the realized IDA1 and DA prices in the training data. Feature engineering reuses the cyclical calendar encoding of Eq. 64 (hour, day-of-week, month) plus DA-price-level features (24-hour rolling mean and standard deviation of the day's DA curve, the same-hour spread from 7 days prior, and the previous hour's spread), and

model selection reuses the walk-forward CV and accuracy-metric machinery of Eq. 68–73 with Naive (zero-spread) as the persistence baseline.

$$\pi_h^{IDA1} = \text{clip}(\pi_h^{DA} + \hat{s}_h, -500, 3000) \quad (89)$$

where $\hat{s}_h$ (EUR/MWh) is the model-predicted spread for hour h , and the result is clipped to the OMIE regulatory price bounds (floor -500 EUR/MWh, ceiling 3000 EUR/MWh).

2A.2 IDA1 Re-Optimization

Source: `phase_2a_ida1_intraday_auction_1/ida1_milp_reoptimiser/ida1_reoptimiser.py`.

IDA1 calls the shared intraday re-optimization engine directly with no IDA1-specific physics: $H^{trade} = H$ (all 24 hours tradable, since no earlier intraday gate has frozen any hours), and the no-churn decision rule of Eq. 40–44 applies unchanged with $\pi_h = \pi_h^{IDA1}$ from Eq. 89.

2A.3 IDA1 Delta Bid Economics

Source: `phase_2a_ida1_intraday_auction_1/ida1_bid_formatting/ida1_bid_formatter.py`, function `format_ida1_bids`.

Each IDA1 order is expressed as a signed volume delta against the DA-committed position, together with its expected revenue impact at the IDA1 price.

$$\delta_h = N_h^{IDA1} \Delta t - N_h^{DA} \Delta t, \quad \rho_h = \delta_h \cdot \pi_h^{IDA1} \quad (90)$$

where δ_h (MWh) is the per-hour volume delta (positive — “buy back” — increasing generation or reducing pumping; negative — “sell down” — the reverse), N_h^{IDA1} , N_h^{DA} (MW) are the IDA1 re-optimized and DA-committed net positions for hour h , and ρ_h (EUR) is the expected revenue impact of that delta at the IDA1 forecast price.

Part 2B — Phase 2B: IDA2 (Second Intraday Auction)

IDA2 closes D-1 22:00 CET and re-optimizes against the IDA1-committed position (not DA), restricted to hours 3–24 — hours 1–2 are frozen because they fall within the delivery window that IDA1’s gate-close no longer allows re-trading (spec INV-11). The pricing model, re-optimization mechanism, and delta-bid economics are structurally identical to IDA1 (Eq. 88–90),

substituting IDA1's committed position for DA's and IDA2-specific training data for the spread model. No new equations are introduced.

Source: phase_2b_ida2_intraday_auction_2/ida2_price_forecasting/ida2_price_forecaster.py (Eq. 88–89, trained on ida2_training_data_2024_2025.xlsx, SIDC IDA2 clearing prices, H3–H24); ida2_milp_reoptimiser/ida2_reoptimiser.py (Eq. 40–44 with $H^{trade} = \{3, \dots, 24\}$ per config/market.yaml gate IDA2.delivery_hours: [3, 24]); ida2_bid_formatting/ida2_bid_formatter.py (Eq. 90, with N_h^{DA} replaced by N_h^{IDA1} , the IDA1-committed position).

Part 2C — Phase 2C: IDA3 (Third Intraday Auction)

IDA3 closes D 10:00 CET (same delivery day, not D-1) and re-optimizes against the IDA2-committed position, restricted to hours 12–24 — hours 1–11 are frozen, confirmed in config/market.yaml: IDA3.delivery_hours: [12, 24] (“hours 1-11 are NOT in the IDA3 product”). A line-by-line diff of ida3_price_forecaster.py, ida3_reoptimiser.py, and ida3_bid_formatter.py against their IDA2 counterparts confirms the formulas are unchanged — only naming, the gate-close time, the training-data file (SIDC IDA3 clearing prices, H12–H24), and the baseline position (IDA2-committed) differ. No new equations are introduced.

Source: phase_2c_ida3_intraday_auction_3/ida3_price_forecasting/ida3_price_forecaster.py (Eq. 88–89, trained on ida3_training_data_2024_2025.xlsx, H12–H24); ida3_milp_reoptimiser/ida3_reoptimiser.py (Eq. 40–44 with $H^{trade} = \{12, \dots, 24\}$); ida3_bid_formatting/ida3_bid_formatter.py (Eq. 90, with N_h^{DA} replaced by N_h^{IDA2} , the IDA2-committed position).

Part 2D — Phase 2D: XBID Continuous Intraday

XBID (SIDC continuous intraday) differs structurally from the IDA auctions: instead of a single gate-close auction, the plant checks the market opportunistically at two windows (W1: D-1 18:30 CET, W2: D 09:30 CET) and may only nudge its position within a small per-order MW cap, since continuous markets trade in small increments rather than clearing a full re-optimization at once. This part documents the genuinely new mechanics: the trade-band constraint, the spread-beating order test, the price proxy (including its synthetic-data generation process), and the open-hours window rule.

2D.1 Per-Order Trade-Band Constraint

Source: `phase_2d_xbid_continuous_intraday/xbid_milp_optimiser/xbid_optimiser.py`, function `optimise_xbid`.

Unlike the IDA gates, which let the shared MILP re-optimize tradable hours freely (subject only to the Part 0 envelope), XBID adds an extra constraint on every open hour limiting how far the position may move from the currently committed net — modeling the reality that continuous-market liquidity only supports small incremental trades, not a full re-clear.

$$N_h - C \leq p_h^{net} \leq N_h + C \quad \forall h \in H^{open} \quad (91)$$

where $H^{open} \subseteq H$ is the set of hours still open at the current check window (Eq. 93), N_h (MW) is the net position committed by the latest prior gate (DA, or the most recent IDA auction if it has run), and C (MW) is the configured per-order volume cap (`xbid_max_volume_per_order_mw`). Hours outside H^{open} remain frozen via Eq. 39, exactly as in the IDA gates.

2D.2 XBID Order Acceptance Test

Source: `xbid_optimiser.py`, function `optimise_xbid`.

An XBID order is placed only if the average expected gain per repositioned MWh beats the configured bid-ask spread threshold — a different no-churn formulation from the IDA gates’ fixed/dynamic EUR threshold (Eq. 43), reflecting that continuous-market profitability is naturally measured per unit of traded volume rather than as a lump-sum improvement.

$$I = \sum_{h \in H^{open}} \pi_h (p_h^{net,new} - N_h) \Delta t, \quad \Phi = \frac{1}{2} \sum_{h \in H^{open}} |p_h^{net,new} - N_h| \Delta t \quad (92)$$

$$\text{decision} = \begin{cases} \text{PLACE ORDERS} & I \geq \sigma_{min} \cdot \max(\Phi, \epsilon) \\ \text{NO_ORDER} & \text{otherwise} \end{cases} \quad (93)$$

where I (EUR) and Φ (MWh) are the same expected-improvement and one-way-volume quantities as Eq. 41–42 (computed here over H^{open} rather than a gate’s full tradable window), σ_{min} (EUR/MWh) is the configured minimum XBID spread (`xbid_min_spread_eur_mwh`), so $\sigma_{min} \cdot \Phi$ is the minimum acceptable total gain at that average spread, and ϵ is a small constant avoiding division by zero when $\Phi \rightarrow 0$.

2D.3 Open-Hours Window Rule

Source: `phase_2d_xbid_continuous_intraday/xbid_price_forecasting/xbid_price_loader.py`, function `tradable_hours_for_window`.

XBID closes trading 1 hour before each delivery period. At check window W1 (the evening before delivery), the entire day is still more than 1 hour away, so all hours are open; at check window W2 (mid-morning of the delivery day), hours up to approximately 10:30 are within or past the 1-hour cutoff.

$$H^{open} = \begin{cases} H & \text{window} = W1 \\ \{h \in H: h \geq 11\} & \text{window} = W2 \end{cases} \quad (94)$$

2D.4 XBID Price Proxy and Window Drift

Source: `xbid_price_forecasting/xbid_price_loader.py`, function `fetch_xbid_prices`; `xbid_price_forecasting/xbid_price_forecaster.py`.

No live XBID order-book feed is available without a commercial EPEX SPOT subscription, so the price is proxied by the same spread-forecasting methodology as the IDA gates (Eq. 88–89, trained on synthetic XBID mid-price data), with a small additional per-window drift layered on top to approximate intra-window order-book movement.

$$\pi_{h,w}^{XBID} = \max(1.0, \pi_h^{XBID,base} + u_{h,w}) \quad (95)$$

where $\pi_{h,w}^{XBID}$ (EUR/MWh) is the XBID proxy price for hour h at check window w , $\pi_h^{XBID,base}$ is the spread-model price from Eq. 89, and $u_{h,w}$ is uniform drift noise on $[-1.5, 1.5]$ EUR/MWh, seeded by window and delivery date (deterministic given those inputs).

2D.5 Synthetic XBID Training Data — OU Spread Process

Source: `xbid_price_forecasting/create_xbid_training_data.py`, function `_daily_ou_spread`.

Because no real XBID order-book history exists for training, the spread-model training labels (Eq. 88) are generated synthetically: an Ornstein-Uhlenbeck mean-reverting process simulates the intraday evolution of the XBID-DA spread around a randomly drawn daily mean level, reset

each day, producing realistic serial correlation and a target spread standard deviation wider than IDA1/2/3 (continuous markets trade closer to delivery with more microstructure noise).

$$L_d \sim \mathcal{N}(0, 11^2), \quad s_{d,1} = L_d + \mathcal{N}(0, \sigma_{OU}^2) \quad (96)$$

$$s_{d,h} = s_{d,h-1} + \theta(L_d - s_{d,h-1}) + \sigma_{OU} \mathcal{N}(0,1) \quad h = 2, \dots, 24 \quad (97)$$

where L_d (EUR/MWh) is the random daily mean spread level for day d , $s_{d,h}$ (EUR/MWh) is the simulated spread at hour h of day d , $\theta = 0.35$ is the mean-reversion speed, and $\sigma_{OU} = 6.5$ EUR/MWh is the hourly innovation standard deviation, calibrated so the combined daily-level-plus-hourly-noise process produces a total spread standard deviation of approximately 14 EUR/MWh. The simulated spread is further adjusted with deterministic seasonal effects (a PV-driven negative spread during summer hours 12–15, additional downward pressure when DA price is negative, and a small positive adjustment during the evening peak hours 19–21) before being added to a separately synthesized DA price to form the training target.

2D.6 Synthetic Day-Ahead Price Generator (Training Data Only)

Source: create_xbid_training_data.py, function _da_price.

The DA price series underlying the synthetic XBID training data is generated independently from the live DA forecaster's fallback curve (Eq. 74) a separate, sinusoidal-shaped synthetic price model used only to construct historical training labels, never called during live forecasting.

$$\pi_h^{synth,train} = \text{clip}(65 + P_h + S_h + W + \xi_d + \epsilon_h, -50, 300) \quad (98)$$

where $P_h = 25\sin\left(\frac{\pi(h-6)}{14}\right)$ for $6 \leq h \leq 20$ (else -5) is a sinusoidal daily peak shape, $S_h = -15\max\left(0, \sin\left(\frac{\pi(h-9)}{8}\right)\right)$ in summer months (else 0) is a midday solar-driven dip, $W = -8$ on weekends (else 0), $\xi_d \sim \mathcal{N}(0, 50^2)$ is a day-level price shock shared by all 24 hours of day d (inducing realistic day-to-day and within-day price correlation), and $\epsilon_h \sim \mathcal{N}(0, 6^2)$ is hourly noise.

2D.7 XBID Order Economics

Source: `phase_2d_xbid_continuous_intraday/xbid_bid_formatting/xbid_bid_formatter.py`, function `format_xbid_orders`.

XBID order sizing and revenue-impact accounting reuse the identical delta-bid economics of Eq. 90 (δ_h, ρ_h), with the baseline N_h^{DA} generalized to “the latest committed position across all prior gates” (DA, or the most recent IDA auction if it has run), and an additional minimum-delta filter suppressing orders below a configurable noise floor (default 0.1 MWh) before submission.

Part 3A — Phase 3A: aFRR Capacity Offer

aFRR (automatic Frequency Restoration Reserve, PICASSO platform, FAT 5 min) capacity offers are sized by the shared reserve-offer builder of Part 0, Section C.10 (Eq. 45–53), with `headroom_fraction = 1.0` since aFRR has first call on the plant’s available headroom ahead of mFRR. This part documents the aFRR-specific capacity-price forecast and its synthetic training-data generation, which differ structurally from the spread model used for energy prices.

3A.1 aFRR Capacity Price Forecast

Source: `phase_3a_afrr_automatic_frequency_reserve/afrr_price_forecasting/afrr_price_forecaster.py`, function `_model_forecast`.

Unlike the energy-price forecasters (Eq. 88–89), which predict a spread added onto the DA price, aFRR capacity prices are forecast directly as two independent price levels — one model for upward capacity, one for downward — since capacity availability payments are not anchored to the DA energy price the way intraday energy prices are.

$$c_h^{up} = \text{clip}(\hat{c}_h^{up}, 0, C_{max}), \quad c_h^{dn} = \text{clip}(\hat{c}_h^{dn}, 0, C_{max}) \quad (99)$$

where c_h^{up}, c_h^{dn} (EUR/MW) are the final forecast upward and downward capacity availability prices for hour h , $\hat{c}_h^{up}, \hat{c}_h^{dn}$ are the raw model predictions (separately trained Naive/Ridge/LightGBM models, selected via the same walk-forward CV machinery as Eq. 72–73, using cyclical calendar features (Eq. 64), the DA price level and its 24-hour rolling mean/std,

and each direction's own 7-day-lagged capacity price as predictors), and $C_{max} = 250$ EUR/MW is the REN regulatory price ceiling.

3A.2 aFRR Offer Sizing and Checking

Source: `phase_3a_afrr_automatic_frequency_reserve/afrr_reserve_offer_builder/afrr_offer_builder.py`, `afrr_offer_checker.py`.

Both modules are thin parameter bindings to the shared reserve-offer engine: `build_afrr_offers` calls Eq. 45–50 with `product="aFRR"`, `fat_min = 5` minutes, the market's configured maximum offer sizes, and `headroom_fraction = 1.0`; `check_afrr_offers` calls the shared checker (Eq. 51–53) with `cap_price_max = 250` EUR/MW (Eq. 99's ceiling). No new equations are introduced.

3A.3 Synthetic aFRR Training Data — OU Capacity Price Process

Source: `afrr_price_forecasting/create_afrr_training_data.py`, functions `_daily_cap_ou`, `generate`.

As with XBID (Eq. 96–97), no real aFRR clearing-price history is available for the full 2019–2025 training window, so capacity prices are synthesized with a daily-reset Ornstein-Uhlenbeck process anchored to a direction-specific mean, then adjusted by a DA-price scarcity signal so upward capacity is priced higher in tight (high-DA-price) hours and downward capacity higher in surplus (low-DA-price) hours — reflecting that TSOs activate upward reserve when supply is scarce and downward reserve when generation is in surplus.

$$B_d \sim \mathcal{N}(\mu, (0.25\mu)^2), \quad u_{d,1} = B_d + \mathcal{N}(0, \sigma^2) \quad (100)$$

$$u_{d,h} = u_{d,h-1} + \theta(B_d - u_{d,h-1}) + \sigma \mathcal{N}(0,1), \quad \theta = 0.40 \quad (101)$$

where B_d (EUR/MW) is the random daily bias level for day d (drawn separately for the up and down directions, with direction-specific mean μ and innovation std σ : $\mu = 22$ (seasonally adjusted, +6 in winter, −3 in summer, −4 on weekends), $\sigma = 5$ for upward capacity; $\mu = 10$, $\sigma = 3$ for downward capacity), and $u_{d,h}$ (EUR/MW) is the simulated base capacity price at hour h , clipped to $[0, C_{max}]$.

$$\kappa_h = \frac{\pi_h^{DA} - \pi_{min}^{DA}}{\max(1, \pi_{max}^{DA} - \pi_{min}^{DA})} \quad (102)$$

$$c_h^{up} = u_{d,h}^{up} + 20 \kappa_h, \quad c_h^{dn} = u_{d,h}^{dn} + 10 (1 - \kappa_h) + 4 \cdot \mathbb{1}_{solar}(h) \quad (103)$$

where $\kappa_h \in [0,1]$ is the hour's scarcity index (Eq. 102), computed from that day's DA price range ($\pi_{min}^{DA}, \pi_{max}^{DA}$ are the day's minimum and maximum simulated DA prices, generated by the same synthetic DA price model as Eq. 98), and $\mathbb{1}_{solar}(h)$ in Eq. 103 is an indicator equal to 1 for hours 11–15 in summer months (added PV-surplus pressure on downward reserve demand), both final prices clipped to $[0, C_{max}]$.

Part 3B — Phase 3B: mFRR Capacity Offer

mFRR (manual Frequency Restoration Reserve, MARI platform, FAT 12.5 min) is sized from whatever headroom aFRR did not take: `build_mfr_offers` calls the shared engine (Eq. 45–50) with `reserved_up/reserved_dn` set to the aFRR commitment (the $R_h^{up,prior}, R_h^{dn,prior}$ terms already defined generically in Eq. 45–46) and `headroom_fraction` = 0.20 (`mfr.max_offer_fraction`), leaving an operating margin rather than committing the entire residual envelope to a single, slower product. `check_mfr_offers` likewise binds directly to the shared checker (Eq. 51–53) with the same `reserved_up/reserved_dn` terms. No new offer-sizing equations are introduced.

The capacity-price forecast (Eq. 99) and its OU-based synthetic training-data generation (Eq. 100–103) are structurally identical to aFRR, confirmed via line-by-line diff, but the codebase is explicit that mFRR pricing is **not** a fixed fraction of aFRR — it is trained as an independent model on its own MARI-anchored synthetic data (REN's MARI accession date is 27 Nov 2024, so training history is limited to roughly 13 months vs. aFRR's 6 years, and cross-validation accordingly uses 3 folds instead of 4). The calibration constants differ:

Constant (Eq. 100–103)	aFRR (Part 3A)	mFRR
Mean reversion θ	0.40	0.35
Daily-bias std factor	0.25μ	0.30μ
μ_{up} (base), seasonal adj.	22, \$6/\$3	9, \$3/\$2
σ_{up}	5.0	3.5
μ_{dn}, σ_{dn}	10, 3.0	7, 2.5

Constant (Eq. 100–103)	aFRR (Part 3A)	mFRR
Scarcity coefficient (up / down)	20 / 10	8 / 4
Solar-hour down adjustment	+4.0	+2.0

Source: phase_3b_mfrr_manual_frequency_reserve/mfrr_price_forecasting/mfrr_price_forecaster.py (Eq. 99, trained on mfrr_training_data_2024_2025.xlsx, 3-fold CV); create_mfrr_training_data.py (Eq. 100–103 with the constants above); mfrr_reserve_offer_builder/mfrr_offer_builder.py, mfrr_offer_checker.py (bindings to Eq. 45–53); mari_mfrr_price_loader.py, run_mfrr.py, mfrr_price_train_val_test.py (no new equations).

Part 4A — Phase 4A: ISP Real-Time Dispatch

Phase 4A turns the committed hourly net position into concrete per-unit PSP setpoints and a BESS trim, simulates actual delivery against the schedule, and tracks the resulting per-ISP deviation that feeds Phase 5C imbalance settlement. This is the first phase whose math is not solved by the shared MILP — it is real-time allocation logic operating on the MILP’s already-committed hourly target.

4A.1 PSP Greedy Unit Allocation

Source: phase_4a_isp_real_time_dispatch/isp_setpoint_dispatch/psp_setpoint_dispatcher.py, function `_spread`.

The MILP optimizes plant-level totals; real-time dispatch must convert an hourly PSP net target into concrete per-unit setpoints honoring each unit’s minimum-stable-load band (Eq. 10) or pump band (Eq. 11). The dispatcher commits the fewest units necessary, distributing the target as evenly as possible across them.

$$k = \min \left(n_U, \max \left(1, \left\lceil \frac{T}{P_{max}} \right\rceil \right) \right) \quad (104)$$

where k is the number of units committed, T (MW) is the magnitude of the PSP net target for the hour (turbine or pump, depending on sign), P_{max} is the per-unit maximum (turbine or pump,

matching T 's mode), and n_U is the total number of PSP units. If distributing T evenly across k units would put each below its minimum stable load P_{min} , k is reduced by one and rechecked, repeating until $T/k \geq P_{min}$ or $k = 1$.

$$p_i = \text{clip}\left(\frac{T}{k}, P_{min}, P_{max}\right), \quad i = 1, \dots, k \quad (105)$$

where p_i (MW) is the per-unit setpoint, assigned to units sequentially with any final sub-minimum remainder folded into the previous unit's setpoint rather than left as an infeasible sliver.

4A.2 BESS Real-Time Trim

Source: `phase_4a_isp_real_time_dispatch/isp_setpoint_dispatch/bess_setpoint_dispatcher.py`, function `setpoint`.

The battery covers the fast residual between the hourly net target and what the allocated PSP units deliver ($r = N_h - \sum_i p_i$, the difference between the committed target and Eq. 105's PSP total), trimmed by both rated power and the energy actually available given current SoC.

$$\begin{aligned} p^{dis} &= \min\left(r, P^{bess}, \frac{\max(0, e - E_{min}) \eta_d}{\Delta t}\right) \quad (r > 0), & p^{chg} \\ &= \min\left(-r, P^{bess}, \frac{\max(0, E_{max} - e)}{\eta_c \Delta t}\right) \quad (r < 0) \end{aligned} \quad (106)$$

where r (MW) is the residual to be covered (positive = discharge, negative = charge), e (MWh) is current BESS stored energy, and the energy-availability terms mirror the SoC continuity relationship of Eq. 26, ensuring the real-time setpoint can never be physically infeasible regardless of what the residual calls for.

4A.3 Simulated Actual Delivery and ISP Deviation Tracking

Source: `phase_4a_isp_real_time_dispatch/telemetry/ren_isp_signal_loader.py`, function `simulate_actual_delivery`; `isp_activation_tracking/isp_position_tracker.py`, function `track`.

Without live SCADA/REN telemetry, actual per-ISP delivery is simulated as the scheduled net position plus small random delivery error, representing unavoidable forecast and control-lag deviations; the resulting deviation is what Phase 5C settles as imbalance.

$$a_{isp} = s_{isp} + v_{isp}, \quad v_{isp} \sim \text{Uniform}(-3, 3) \text{ MW} \quad (107)$$

where a_{isp} , s_{isp} (MW) are actual and scheduled net position for ISP isp , and v_{isp} is delivery noise seeded deterministically by delivery date.

$$d_{isp} = a_{isp} - s_{isp}, \quad \bar{d} = \frac{1}{n} \sum_{isp} |d_{isp}|, \quad D = \sum_{isp} |d_{isp}| \Delta t_{isp} \quad (108)$$

where d_{isp} (MW) is the per-ISP deviation, $\bar{d}$ (MW) is the mean absolute deviation across n ISPs (a delivery-quality KPI), and D (MWh) is the total absolute deviation volume, the quantity that feeds the imbalance settlement calculation in Part 5C.

Part 4B — Phase 4B: aFRR Activation Response

Phase 4B handles TSO-instructed aFRR activations during delivery. Every file in this phase is a thin binding to the shared activation engine documented in Part 0

(`common_layer/optimisation_model/reserve_activation.py`, function `simulate_and_log_activation`, which determines activation pricing and applies the ramp-corrected effective energy of Eq. 56–57) or a pure data read-back. No new equations are introduced.

Source: `phase_4b_afrr_activation_response/afrr_setpoint_dispatch/afrr_activation_handler.py` (calls the shared engine with `fat_min = 5` minutes); `afrr_activation_tracking/afrr_activation_logger.py` (reads back logged activations for display and Phase 5B settlement, no computation); `run_afrr_activation.py` (orchestration only).

Part 4C — Phase 4C: mFRR Activation Response

Structurally identical to Phase 4B, confirmed via diff: the same shared activation engine (Eq. 56–57) is called with `fat_min = 12.5` minutes (`cfg.market.mfrr.fat_min`) in place of aFRR's 5 minutes. No new equations.

Source: phase_4c_mfrr_activation_response/mfrr_setpoint_dispatch/mfrr_activation_handler.py, mfrr_activation_tracking/mfrr_activation_logger.py, run_mfrr_activation.py.

Part 5A — Phase 5A: DA/IDA Energy Settlement

Phase 5A computes actual post-delivery energy revenue from the committed positions across all gates, valued at final OMIE settlement prices (as opposed to the forecast prices used during bidding). This is the first genuinely new revenue-accounting logic in the document — distinct from the objective function’s energy-revenue term (Eq. 2), which values a forecast schedule, not a settled one.

5A.1 Day-Ahead Settlement

Source: phase_5a_da_ida_settlement/energy_settlement_calculation/da_settlement_calculator.py, function settle_da.

$$R_{DA} = \sum_{h \in H} V_h^{DA} \cdot \pi_h^{settle} \quad (109)$$

where R_{DA} (EUR) is total DA settlement revenue, V_h^{DA} (MWh) is the committed DA volume for hour h (positive for net sell, negative for net buy/pumping), and π_h^{settle} (EUR/MWh) is the final OMIE settlement price for hour h (falling back to the originally bid price if a settlement price is unavailable).

5A.2 Intraday Delta Settlement

Source: phase_5a_da_ida_settlement/energy_settlement_calculation/ida_settlement_calculator.py, function settle_intraday.

Each intraday gate (IDA1, IDA2, IDA3, XBID) is settled only on the volume it *changed* relative to the position committed by the prior gate in sequence — not on its full position — so that walking the gate chain DA → IDA1 → IDA2 → IDA3 → XBID accounts for the total traded volume exactly once per gate, with no double-counting of hours that were not re-traded at a given gate.

$$\Delta V_h^g = V_h^g - V_h^{g-1}, \quad R_g = \sum_h \Delta V_h^g \cdot \pi_h^{settle,g} \quad (110)$$

where g indexes the intraday gates in chain order, V_h^g (MWh) is the committed volume at gate g for hour h (with V_h^{g-1} the volume committed by the previous gate in the chain, V_h^{DA} for $g = \text{IDA1}$), ΔV_h^g (MWh) is the volume delta traded at gate g , $\pi_h^{settle,g}$ (EUR/MWh) is gate g 's settlement price, and R_g (EUR) is gate g 's settlement revenue. Total intraday revenue is $\sum_g R_g$.

Part 5B — Phase 5B: Reserve Settlement

Reserve settlement has two independent revenue streams — capacity (availability) payments owed regardless of activation, and activation (energy) payments owed only for MW actually called by the TSO. A single product-agnostic function serves both aFRR and mFRR settlement; `settle_mfrr` is a direct call into the same calculator `settle_afrr` uses, confirmed by reading both files.

5B.1 Capacity (Availability) Settlement

Source: `phase_5b_reserve_settlement/reserve_settlement_calculation/afrr_settlement_calculator.py`, function `settle_reserve`.

$$R_{cap} = \sum_h (o_h^{up} \cdot c_h^{up} + o_h^{dn} \cdot c_h^{dn}) \quad (111)$$

where R_{cap} (EUR) is total capacity settlement revenue, o_h^{up} , o_h^{dn} (MW) are the offered up/down capacity for hour h (Eq. 49–50), and c_h^{up} , c_h^{dn} (EUR/MW) are the cleared capacity prices — paid for the full hour regardless of whether the TSO ever calls the reserve.

5B.2 Activation (Energy) Settlement

Source: `afrr_settlement_calculator.py`, function `settle_reserve`.

$$R_{act} = \sum_a (u_a \cdot \tau_a \cdot \pi_a^{up} + d_a \cdot \tau_a \cdot \pi_a^{dn}) \quad (112)$$

where R_{act} (EUR) is total activation settlement revenue, the sum runs over every logged activation row a , u_a , d_a (MW) are the up and down MW activated in that row, π_a^{up} , π_a^{dn}

(EUR/MWh) are that activation's separately-stored up and down prices (since up and down activations settle at different rates — a scarcity premium versus a discount relative to the DA price, per Part 0 Eq. 99's pricing logic), and τ_a (h) is the ramp-corrected effective ISP duration of Eq. 57 when stored with the activation row, falling back to the face ISP duration otherwise.

Total reserve settlement revenue for a product is $R_{cap} + R_{act}$.

Part 5C — Phase 5C: Imbalance Settlement

Imbalance is the uninstructed deviation between actual and scheduled delivery — deliberately excluding TSO-instructed reserve activations, which are fully settled separately in Part 5B. Portugal applies dual imbalance pricing: a long (over-delivered) position is sold at a discount, a short (under-delivered) position is bought back at a premium, creating an incentive to deliver exactly the committed schedule.

5C.1 Imbalance Volume

Source:

phase_5c_imbalance_settlement/imbalance_price_and_volume/imbalance_volume_calculator.py,
function compute_imbalance.

$$m_{isp} = a_{isp} - s_{isp}, \quad M_{isp} = m_{isp} \cdot \Delta t_{isp} \quad (113)$$

where m_{isp} (MW) is the uninstructed imbalance power for ISP isp (the same $a_{isp} - s_{isp}$ quantity as Eq. 108's deviation, deliberately not adjusted for reserve activations since those are excluded from the simulated actual delivery and settled independently in Part 5B), and M_{isp} (MWh) is the imbalance volume; positive (long) means more delivered than scheduled, negative (short) means less.

5C.2 Dual Imbalance Pricing — Fallback Prices

Source: imbalance_price_and_volume/ren_imbalance_price_loader.py, function
fetch_imbalance_prices.

When live REN DataHub ISP prices are unavailable, short and long imbalance prices are derived from the DA price forecast (Eq. 88's π_h^{DA} , or Eq. 74's fallback) via configured multipliers.

$$\pi_h^{short} = \pi_h^{DA} \cdot \phi_{short}, \quad \pi_h^{long} = \pi_h^{DA} \cdot \phi_{long} \quad (114)$$

where π_h^{short} , π_h^{long} (EUR/MWh) are the short (premium) and long (discount) fallback imbalance prices, and $\phi_{short} = 1.20$, $\phi_{long} = 0.85$ are the configured fallback multipliers (market.imbalance.fallback_short_factor, fallback_long_factor).

5C.3 Imbalance Settlement

Source: imbalance_settlement_calculation/imbalance_settlement_calculator.py, function settle_imbalance.

$$R_{long} = \sum_{isp: M_{isp} > 0} M_{isp} \cdot \pi_{h(isp)}^{long}, \quad C_{short} = \sum_{isp: M_{isp} < 0} |M_{isp}| \cdot \pi_{h(isp)}^{short} \quad (115)$$

$$R_{imb} = R_{long} - C_{short} \quad (116)$$

where R_{long} (EUR) is revenue from long (over-delivered) ISPs at the discounted long price, C_{short} (EUR) is the cost of buying back short (under-delivered) ISPs at the premium short price, $h(isp)$ maps an ISP to its containing hour (Eq. via date_utils.isp_to_hour), and R_{imb} (EUR) is net imbalance settlement — typically negative under dual pricing, which is precisely the financial incentive for the plant to track its committed schedule closely.

Part 5D — Phase 5D: Analytics and Reporting

Phase 5D consolidates the settlement streams of Parts 5A–5C into a single daily P&L, derives operational and economic KPIs from the solved schedule, and exports everything to the 5-sheet Excel report. Risk metrics (VaR, CVaR, Sharpe ratio) are **not** computed here — they belong to Phase 6 Backtesting (risk_analytics/portfolio_risk_metrics.py), documented in Part 6 below.

5D.1 Daily P&L Aggregation

Source: phase_5d_analytics_and_reporting/analytics_and_kpis/daily_pnl_calculator.py, function compute_daily_pnl.

$$\Pi = R_{DA} + \sum_g R_g + R_{tot}^{aFRR} + R_{tot}^{mFRR} + R_{imb} \quad (117)$$

where Π (EUR) is total daily P&L, R_{DA} is Eq. 109, $\sum_g R_g$ is the summed intraday delta revenue of Eq. 110 (IDA1+IDA2+IDA3+XBID), R_{tot}^{aFRR} , R_{tot}^{mFRR} are each product's $R_{cap} + R_{act}$ from Eq. 111–112, and R_{imb} is Eq. 116.

5D.2 Revenue Share by Stream

Source: analytics_and_kpis/revenue_breakdown_analyzer.py, function revenue_shares; analytics_and_kpis/operational_analytics.py, function compute_economic_kpis_extended (DA/FRR split).

$$s_k = \frac{100 \cdot \max(0, \Pi_k)}{\sum_{j: \Pi_j > 0} \Pi_j} \quad (118)$$

where s_k (%) is revenue stream k 's share of gross positive revenue (only positive-revenue streams contribute to the denominator, so a net imbalance cost is reported separately rather than distorting the shares of the streams that actually earned).

5D.3 Capacity Factors and PV Utilization

Source: operational_analytics.py, function compute_economic_kpis_extended.

$$CF_x = \frac{100 \cdot E_x}{P_x \cdot n \cdot \Delta t} \quad (119)$$

where CF_x (%) is the capacity factor for asset $x \in \{\text{turbine, pump, BESS discharge}\}$, E_x (MWh) is that asset's total delivered energy over the horizon, P_x (MW) is its rated capacity, and n is the number of hours in the horizon.

$$U_{PV} = \frac{100 \cdot \sum_h p_h^{pv}}{\sum_h p_h^{av}} \quad (120)$$

where U_{PV} (%) is PV utilization — the share of physically available solar energy (Eq. 58) actually delivered to the grid rather than curtailed (Eq. 28).

5D.4 Reservoir Fill Percentage

Source: operational_analytics.py, function compute_economic_kpis_extended; restated for display in daily_excel_reports/dispatch_sheet_builder.py.

$$F_{res} = \frac{100(v_{h_N}^{up} - V_{min}^{up})}{V_{max}^{up} - V_{min}^{up}} \quad (121)$$

where F_{res} (%) is the upper reservoir's fill level at the end of the horizon, expressed as a fraction of its usable range (Eq. 31's bounds).

5D.5 Price-Dispatch Correlation

Source: operational_analytics.py, function _correlation.

A standard Pearson correlation coefficient quantifies how closely turbine dispatch tracks the DA price signal across the day — a sanity diagnostic that the optimizer is genuinely price-responsive rather than dispatching on a fixed pattern.

$$\rho = \frac{\sum_h (\pi_h - \bar{\pi}) (p_h^{trb} - \bar{p}^{trb})}{\sqrt{\sum_h (\pi_h - \bar{\pi})^2} \sqrt{\sum_h (p_h^{trb} - \bar{p}^{trb})^2}} \quad (122)$$

where $\rho \in [-1,1]$ is the Pearson correlation coefficient between hourly DA price and total turbine power, $\bar{\pi}$, $\bar{p}^{trb}$ are their means across the horizon, and $p_h^{trb} = \sum_u p_{u,h}^{trb}$ is fleet turbine power for hour h .

The remaining functions in this phase (compute_operational_patterns, compute_temporal_patterns, compute_frr_strategy_metrics, and the Excel sheet builders) compute descriptive counts, averages, and run-length statistics from already-defined quantities — run durations, peak/off-peak hour counts against price quartiles, time-of-day banding, and volume-weighted average prices restating Eq. 109–116 for spreadsheet display — without introducing further distinct equations.

Part 6 — Phase 6: Backtesting and Validation

Phase 6 replays a span of historical delivery days through the full pipeline, validates forecast accuracy and MILP solution quality, and computes the portfolio-level risk metrics used in

trading-desk risk reporting. This part contains the document’s final new material: forecast error metrics, synthetic historical-realization generation, and the complete VaR/CVaR/Sharpe/drawdown risk framework.

6.1 Forecast Accuracy — MAPE

Source:

phase_6_backtesting_and_validation/forecast_and_model_validation/price_forecast_validator.py,
function error_metrics.

In addition to MAE and RMSE (Eq. 68–69, reused here unchanged), backtesting adds Mean Absolute Percentage Error for both DA price and PV forecast validation.

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{\hat{y}_i - y_i}{y_i} \right| \quad (123)$$

where the sum excludes any sample with $|y_i| < 10^{-9}$ to avoid division by zero (a NaN-safe convention noted directly in the source).

6.2 Synthetic Historical Realization

Source: backtest_engine/historical_data_loader.py, function realised_from_forecast.

Without archived OMIE/REN history, the “actual” realized series used to validate forecast accuracy during backtesting is synthesized as the forecast perturbed by a bounded relative error, deterministic per delivery date.

$$y_h = \hat{y}_h(1 + \epsilon_h), \quad \epsilon_h \sim \text{Uniform}(-\rho, \rho) \quad (124)$$

where y_h is the synthetic realized value, $\hat{y}_h$ is the forecast value being validated, and $\rho = 0.10$ (10%) is the configured relative error bound.

6.3 Portfolio Risk Metrics

Source: phase_6_backtesting_and_validation/risk_analytics/portfolio_risk_metrics.py, functions historical_var, historical_cvar, bootstrap_var_cvar, sharpe_ratio, max_drawdown.

These five functions compute the complete risk framework from the daily P&L series (Eq. 117) produced by replaying the backtest window, implemented in pure Python with no numpy/scipy dependency.

Historical Value-at-Risk is the $(1 - \alpha)$ -th lower-tail percentile of the daily P&L distribution — the loss level not expected to be exceeded on more than $(1 - \alpha)$ of days.

$$\text{VaR}_\alpha = \Pi_{(\lceil(1-\alpha)n\rceil)} \quad (125)$$

where $\Pi_{(1)} \leq \Pi_{(2)} \leq \dots \leq \Pi_{(n)}$ is the sorted daily P&L series of n days, and $\alpha \in \{0.95, 0.99\}$ is the confidence level (95% and 99% are both computed).

Conditional VaR (Expected Shortfall) is the mean P&L of every day at or below the VaR threshold — the average outcome conditional on being in the worst tail.

$$\text{CVaR}_\alpha = \frac{1}{|\mathcal{T}_\alpha|} \sum_{d \in \mathcal{T}_\alpha} \Pi_d, \quad \mathcal{T}_\alpha = \{d: \Pi_d \leq \text{VaR}_\alpha\} \quad (126)$$

Monte Carlo bootstrap resamples the P&L series with replacement to build a confidence interval on the VaR/CVaR point estimates — necessary because a short backtest window (e.g., 30 days) is a small sample for tail-risk estimation.

$$\text{VaR}_{mean}^{boot} = \frac{1}{B} \sum_{b=1}^B \text{VaR}_\alpha(\Pi^{(b)}), \quad \text{VaR}_{std}^{boot} = \sqrt{\frac{1}{B} \sum_{b=1}^B (\text{VaR}_\alpha(\Pi^{(b)}) - \text{VaR}_{mean}^{boot})^2} \quad (127)$$

where $\Pi^{(b)}$ is the b -th bootstrap resample (drawn with replacement, same size n as the original series), $B = 10,000$ is the configured number of resamples, and an identical pair of formulas (substituting CVaR_α for VaR_α) gives $\text{CVaR}_{mean}^{boot}$, CVaR_{std}^{boot} .

Annualized Sharpe ratio assumes a zero risk-free rate, standard practice for short-term energy trading.

$$S = \frac{\bar{\Pi}}{\sigma_\Pi} \sqrt{252} \quad (128)$$

where $\bar{\Pi}$, σ_{Π} are the mean and sample standard deviation ($n - 1$ denominator) of the daily P&L series, and 252 is the assumed number of trading days per year.

Maximum drawdown is the largest peak-to-trough decline in cumulative P&L over the backtest window.

$$C_d = \sum_{i=1}^d \Pi_i, \quad MDD = \max_d \left(\max_{i \leq d} C_i - C_d \right) \quad (129)$$

where $C_d(EUR)$ is cumulative P&L through day d , and $MDD(EUR)$ is the maximum drawdown — the largest gap between any running peak and a subsequent trough in the cumulative curve, reported as a positive number.

The MILP solution-quality checker

(forecast_and_model_validation/milp_solution_quality_checker.py) and the backtest runner/exporter (backtest_engine/backtest_runner.py, backtest_excel_reports/backtest_report_exporter.py, run_backtest.py) call already-documented functions from Part 0 and Part 5D and introduce no further equations.

Document Summary

This document extracts **129 equations** from the complete production codebase, organized into Part 0 (Common Layer, 63 equations) and Parts 1 through 6 (phase-specific extensions and reuses, 66 equations), with every equation traced to its source file and function and cross-referenced where one part reuses or extends another. Parts confirmed as pure structural reuse with zero new equations (2B, 2C, 3B, 4B, 4C) demonstrate the codebase’s stated design principle — “the physics lives once” — holding true under direct line-by-line and diff-based verification across all 15 phases and the shared common layer.